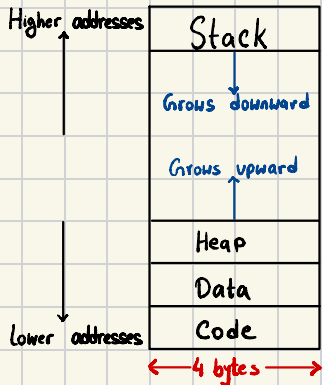


Memory Layout



At runtime, the loader tells OS to give a big blob of memory to the program.

On a 32 bit system, the memory has 32 bit addresses

1 address = 1 byte
Therefore, we have 2^{32} bytes of memory

Registers

- i) Registers are located on CPU.
- ii) Referred to by names (ebp, esp, eip), not addresses.

In each row of grid we put 4 bytes = 1 word

address 0xFFFFFFFF

least Significant byte (LSB) ↑				Most significant byte (MSB) ↑
	0xDE	0xAD	0xBE	0xEF
	0x11	0x22	0x33	0x44

address 0x00000004

address 0x00000000

address 0x00000001

Byte at address 0x00000000? You Should Say 0x11

Word at address 0x00000000? 0x44332211

How do we write dates?

Year - Month - Day

2026 - 01 - 02

Big endian, because we start with the biggest unit.

Day - month - year

02 - 01 - 2026

Little endian, because we start with the smallest unit.

Memory is storing words in "backwards" order.

Calling a function

Before Function call

```
int main(void) {  
    Vulnerable();  
    return 0;  
}
```

Caller

During Function call

```
Void vulnerable(void) {  
    Char name[20];  
    gets(name);  
}
```

Callee

After Function returns

```
int main(void) {  
    Vulnerable();  
    return 0;  
}
```

Caller

The Caller function (main) calls the Callee function (Vulnerable)

The Callee function executes and then returns control to the Caller function.

Before Function Call

```
int main(void) {  
    Vulnerable();  
    return 0;  
}
```

Caller

During Function Call

```
void vulnerable(void) {  
    char name[20];  
    gets(name);  
}
```

Callee

After function returns

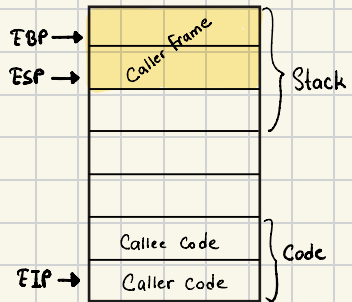
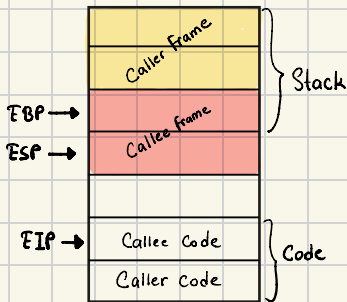
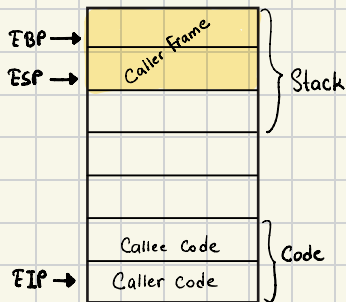
```
int main(void) {  
    Vulnerable();  
    return 0;  
}
```

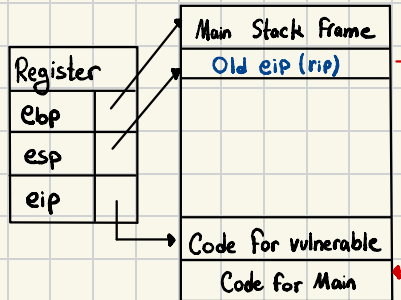
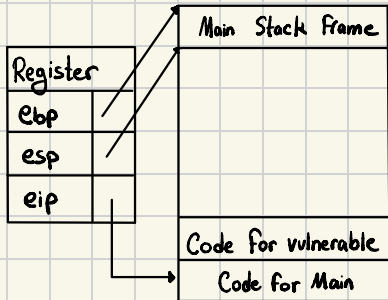
Caller

EBP register points to the top of the Stack

ESP register points to the bottom of the Stack.

EIP register points to the next instruction to be executed.





Main :

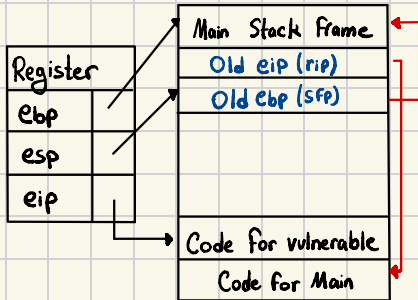
...

Call vulnerable;

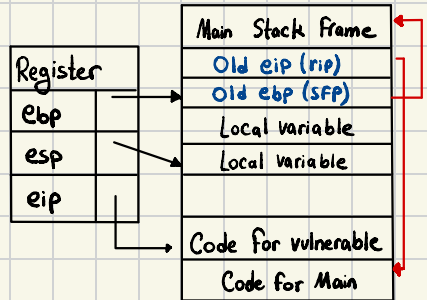
return 0;

The call instruction does 2 things:

- ① It pushes the Current EIP (the address of next instruction in Caller) on the Stack. The Saved EIP is called the RIP (return instruction pointer)
- ② It changes the EIP to point to the instructions of Callee.



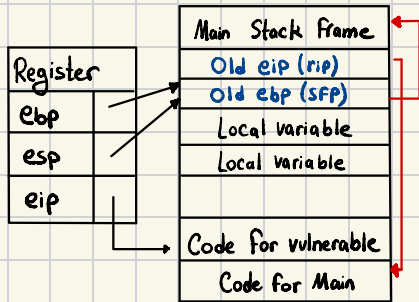
①



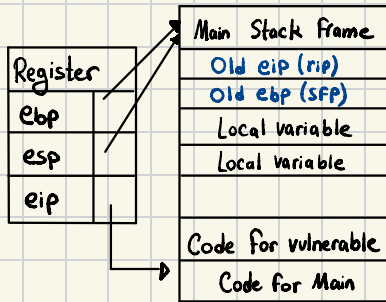
③

These 3 instructions are called Function Prologue. They appear at the start of every function. These 3 instructions create the Stack frame.

Vulnerable :	Comments
① push ebp	We need to restore the value of ebp when returning, So we push the current ebp to stack. The saved value of EBP is known as Stack frame pointer (SFP)
② Move ebp to esp	This instruction moves the EBP down to where ESP is located
③ Move esp	This instruction moves the ESP down to create space for a new stack frame.



①



③

These 3 instructions are called function epilogue. They appear at the end of every function. These 3 functions restore the Caller's Stack Frame.

Vulnerable :	Comments
① Move esp to ebp	This effectively deletes the Space allocated for Callee stack Frame
② Pop ebp	The pop instruction puts the SFP back in EBP. It also increments ESP to delete the popped SFP from Stack.
③ ret	This instruction pops the eip from stack. It puts the RIP to the EIP, which returns program execution to the Caller. It also increments ESP to delete Popped RIP from Stack.

Walking through a buffer overflow

Stack Smashing is the most common kind of buffer overflow.

It occurs in Stack Memory.

What are some values that an attacker can overflow in stack?

- ① Local variables ② Function arguments ③ Saved frame pointer (SFP)
- ④ Return instruction pointer.

Recall: When returning from a program, the EIP is set to the value of the RIP saved in memory. Like the function pointer, this lets attacker choose an address to jump (return) to!

The most common way of executing malicious code is to place it in memory.

Input: **SHELLCODE** + 'A' * 12 + '\x40\xcd\xff\xbf'

12 byte Shellcode + 12 bytes of garbage + Address of where we place Shellcode

Assume we have 12 bytes of Shellcode.

Raw byte: $\text{len}('\text{\textbackslash}\text{ff}') = 1$

Malicious code inserted by attacker into memory, to be executed using a memory safety exploit. It's called a Shellcode because it spawns a terminal, could also delete files, run another program, etc.

Putting together an attack

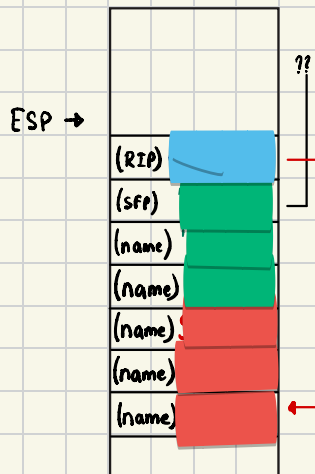
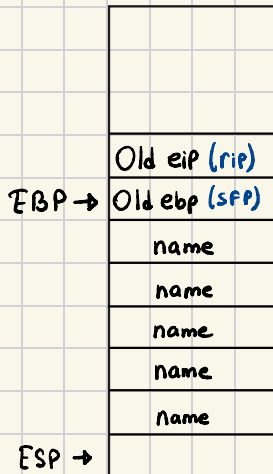
- ① Find a memory safety (e.g buffer overflow) vulnerability.
- ② Write malicious Shell code at a known memory address
- ③ Overwrite the RIP with the address of the Shell code
- ④ Return from function
- ⑤ Begin executing malicious Code.

```

Void vulnerable (void) {
    Char name[20];
    gets (name);
}

int main (Void) {
    Vulnerable ();
    return 0;
}

```



SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf'

12 byte ShellCode + 12 bytes of garbage + Address of where we place Shell Code